Spring Boot

Objective

Understand Spring Boot fundamentals and learn to build, configure, test and maintain REST-based backend applications using Spring Boot.

Outline

• Introduction to Spring Boot
• Project Setup & Fundamentals
• Core Spring Concepts
• Building REST APIs
• Data Access
• Configuration & Profiles
• Testing

Introduction

Spring Boot simplifies Spring application development using starter dependencies, auto-configuration and embedded servers.

Introduction to Spring Boot

Spring vs Spring Boot

Spring Framework is a comprehensive Java framework that provides features such as Dependency Injection (DI), Aspect-Oriented Programming (AOP), transaction management, web development, security, and data access. While powerful, configuring a Spring application traditionally required significant XML or Java-based configuration.

Spring Boot is built on top of the Spring Framework to simplify application development. It eliminates much of the boilerplate configuration by providing auto-configuration, starter dependencies, and embedded servers. Developers can quickly create production-ready applications with minimal setup.

Key features and benefits

Spring Boot provides several features that improve developer productivity:

- **Auto-Configuration** automatically configures Spring components based on the project's dependencies.
- **Starter Dependencies** simplify dependency management by grouping commonly used libraries.
- **Embedded Web Server** allows applications to run without deploying to an external server.
- **Production-ready Features** such as health endpoints and metrics through Spring Boot Actuator.
- **Externalized Configuration** enables different configurations for different environments.

These features help developers spend more time writing business logic instead of framework configuration.

Convention over configuration

One of Spring Boot's core principles is **Convention over Configuration**.

Instead of requiring developers to configure every component manually, Spring Boot provides sensible defaults that work for most applications. Configuration is only needed when the default behavior must be customized.

For example, simply adding the spring-boot-starter-web dependency automatically configures:

- Embedded Tomcat
- Spring MVC
- Jackson for JSON serialization
- Default error handling

without writing any configuration code.

Benefits

- Faster development
- Reduced boilerplate
- Consistent project structure
- Easier maintenance

Project Setup & Fundamentals

Setting up a Spring Boot project correctly is the first step toward building a maintainable and scalable application. Spring Boot simplifies project creation by providing standard project structures, dependency management, and built-in configuration mechanisms. Developers can quickly bootstrap a project using **Spring Initializr** and choose either **Maven** or **Gradle** as the build tool.

A typical Spring Boot project consists of source code, configuration files, dependency definitions, and test classes organized in a standard directory structure. Following this structure helps teams maintain consistency across projects and makes onboarding easier.

Maven / Gradle Setup:

Spring Boot supports both **Maven** and **Gradle** as build automation tools. These tools manage project dependencies, compile source code, execute tests, and package the application.

The easiest way to create a Spring Boot project is through **Spring Initializr**, which generates a project with the required folder structure and dependencies.

A standard Spring Boot project contains the following directories:

```
None
spring-boot-employee-management-demo
├── src
│   ├── main
│   │   ├── java
│   │   └── resources
│   └── test
├── pom.xml / build.gradle
```

- **src/main/java** – Application source code
- **src/main/resources** – Configuration files (`application.properties` or `application.yml`)
- **src/test** – Unit and integration tests
- **pom.xml / build.gradle** – Dependency and build configuration

Both Maven and Gradle are widely used within the Java ecosystem. Teams generally standardize on one build tool for consistency across projects.

Spring Boot Starters:

Managing individual library dependencies can quickly become complex. Spring Boot solves this by providing **Starter Dependencies**, which bundle commonly used libraries required for a specific feature.

Instead of manually adding multiple dependencies, developers include a single starter dependency, and Spring Boot manages compatible library versions automatically.

Some commonly used starters include:

Starter	Purpose
`spring-boot-starter-web`	Build REST APIs and web applications
`spring-boot-starter-data-jpa`	Database access using Spring Data JPA
`spring-boot-starter-validation`	Request validation using Jakarta Bean Validation
`spring-boot-starter-test`	Unit and integration testing support

For example, adding the **Web Starter** automatically includes Spring MVC, embedded Tomcat, JSON support, and other required web libraries.

This approach reduces dependency conflicts and simplifies project configuration.

`@SpringBootApplication` and Auto-Configuration

Every Spring Boot application starts from a class annotated with **`@SpringBootApplication`**.

```
None
// Marks this as the main Spring Boot application class.

// It combines @Configuration, @EnableAutoConfiguration

// and @ComponentScan into a single annotation.

@SpringBootApplication

public class MemberServiceApplication {

    public static void main(String[] args) {

        // Starts the Spring Boot application.

        // Creates the Application Context, performs component scanning,

        // configures beans and starts the embedded server.

        SpringApplication.run(MemberServiceApplication.class, args);

    }

}
```

The `@SpringBootApplication` annotation marks this as the main entry point of a Spring Boot application. When the application starts, execution begins from the `main()` method, where `SpringApplication.run()` is invoked. This method bootstraps the application by creating the Spring Application Context, scanning the project for Spring-managed components, applying auto-configuration, and starting the embedded web server if required.

Instead of requiring developers to configure these components manually, Spring Boot performs most of the setup automatically based on the project's dependencies. This significantly reduces boilerplate configuration and allows developers to focus on implementing business logic.

The `@SpringBootApplication` annotation itself is a combination of three commonly used Spring annotations:

- @Configuration – Marks the class as a source of bean definitions.
- @EnableAutoConfiguration – Automatically configures Spring components based on project dependencies.

- @ComponentScan – Scans the application packages to discover Spring-managed beans.

One of Spring Boot's biggest advantages is **Auto-Configuration**. Instead of manually configuring every component, Spring Boot analyzes the dependencies available in the project and automatically configures the required infrastructure.

For example:

- Adding **spring-boot-starter-web** automatically configures Spring MVC and an embedded Tomcat server.
- Adding **spring-boot-starter-data-jpa** configures JPA, transaction management, and database integration.
- Adding **spring-boot-starter-validation** enables Bean Validation support without additional configuration.

Developers can override the default behavior whenever customization is required, but in most cases the default configuration is sufficient.

Core Spring Concepts

Spring Boot is built on top of the Spring Framework, and understanding its core concepts is essential for developing scalable and maintainable applications. The Spring Framework promotes **loose coupling**, **modularity**, and **testability** by managing application components and their dependencies. Instead of manually creating and connecting objects, Spring uses the **Inversion of Control (IoC)** principle and **Dependency Injection (DI)** to manage the lifecycle of application components.

IoC (Inversion of Control) and Dependency Injection

Inversion of Control (IoC) is a design principle where the control of object creation and dependency management is transferred from the application to the Spring Framework. Rather than creating objects manually using the new keyword, developers define components, and Spring is responsible for creating and managing them.

Dependency Injection (DI) is the mechanism through which Spring provides the required dependencies to a class. This keeps components loosely coupled, making applications easier to maintain, test, and extend.

Spring supports three types of dependency injection:

- Constructor Injection (Recommended)
- Setter Injection
- Field Injection

Among these, **Constructor Injection** is considered the best practice because it makes dependencies explicit, encourages immutability, and simplifies unit testing.

Example

```java
@Service // Marks this class as a Spring-managed Service bean.
public class MemberService {

    // Repository dependency required by this service.
    // Using 'final' ensures the dependency cannot be reassigned.
    private final MemberRepository repository;

    // Spring automatically injects the required MemberRepository bean
    // when creating the MemberService bean.
    public MemberService(MemberRepository repository) {
        this.repository = repository;
    }

    // Business methods use the repository to perform database operations.
}
```

In this example, `MemberService` is annotated with `@Service`, which tells Spring to treat it as a managed bean. During application startup, Spring scans the project, detects this annotation, and creates an instance of the service.

The `MemberService` depends on `MemberRepository` to perform database operations. Instead of creating the repository manually using the `new` keyword, the dependency is provided through the constructor. When Spring creates the `MemberService` bean, it first looks for a `MemberRepository` bean in the Application Context and injects it into the constructor automatically. This process is called **Constructor Injection**.

By allowing Spring to manage dependencies, the service remains loosely coupled to the repository implementation. This makes the application easier to maintain, extend, and test. During unit testing, for example, the actual repository can easily be replaced with a mock implementation without changing the service code.

Benefits of Dependency Injection

- Promotes loose coupling between components.
- Improves code maintainability.
- Simplifies unit testing using mock objects.
- Makes dependencies explicit and easier to understand.

Beans and Application Context

A **Spring Bean** is any object that is created, configured, and managed by the Spring IoC Container. Beans form the building blocks of a Spring application and represent components such as services, repositories, controllers, and configuration classes.

The **Application Context** is the central container responsible for managing these beans. During application startup, Spring scans the project for eligible components, creates the required bean instances, resolves their dependencies, and manages their lifecycle throughout the application's execution.

The lifecycle typically consists of:

- Bean creation
- Dependency injection
- Initialization
- Ready for use
- Destruction during application shutdown

Developers rarely interact with the Application Context directly, as Spring automatically manages the required beans behind the scenes.

Common Spring Annotations

Spring provides several annotations that define the role and behavior of different application components. Understanding these annotations is essential when building Spring Boot applications.

Annotation	Purpose
`@Component`	Generic Spring-managed component.
`@Service`	Represents the business logic layer.
`@Repository`	Represents the data access layer and enables exception translation.
`@RestController`	Creates REST API endpoints and automatically returns JSON responses.
`@Configuration`	Defines configuration classes and bean definitions.
`@Bean`	Registers a manually created object as a Spring Bean.
`@Autowired`	Injects required dependencies (constructor injection is preferred).

Each annotation clearly communicates the responsibility of a class, making the application easier to understand and maintain.

Example

```java
// Repository layer responsible for interacting with the database.
@Repository
```

```
public class MemberRepository {

    // Database operations such as save(), findById(), etc.
}

// Service layer containing the business logic.
@Service
public class MemberService {

    private final MemberRepository repository;

    // Spring injects the repository bean automatically.
    public MemberService(MemberRepository repository) {
        this.repository = repository;
    }
}

// REST Controller exposing HTTP endpoints.
@RestController
@RequestMapping("/members")
public class MemberController {

    private final MemberService service;

    // Spring injects the service bean automatically.
    public MemberController(MemberService service) {
        this.service = service;
    }
}
```

This example demonstrates the layered architecture commonly followed in Spring Boot applications. Each layer has a specific responsibility, making the application easier to understand and maintain.

The `MemberController` acts as the entry point for client requests. When a request is received, the controller delegates the processing to the `MemberService`, which contains the application's business logic. If data needs to be stored or retrieved, the service communicates with the `MemberRepository`. The repository is responsible for interacting with the database and hiding the persistence details from the rest of the application.

Notice that none of these classes create their own dependencies. Spring detects the `@Repository`, `@Service`, and `@RestController` annotations during application startup, creates the required beans, and injects them automatically through constructor injection. This keeps the layers loosely coupled and allows each component to focus on a single responsibility.

Best Practices

- Use the appropriate stereotype annotation (@Service, @Repository, etc.) instead of the generic @Component whenever possible.
- Prefer Constructor Injection over Field Injection.
- Keep each component focused on a single responsibility.

Building REST APIs

REST (Representational State Transfer) is one of the most widely used architectural styles for building web services. Spring Boot provides comprehensive support for developing RESTful APIs through Spring MVC, allowing developers to expose business functionality over HTTP using simple annotations.

In a typical Spring Boot application, REST APIs follow a layered architecture where the **Controller** handles incoming HTTP requests, the **Service** contains business logic, and the **Repository** interacts with the database. This separation of responsibilities improves maintainability, testability, and scalability.

Controllers and Request Mappings

A **Controller** acts as the entry point for client requests. Spring Boot uses the `@RestController` annotation to identify REST controllers, while mapping annotations such as `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@DeleteMapping` define how HTTP requests are routed to specific methods.

Common request mapping annotations include:

- `@RequestMapping` – Defines the base URL for a controller.
- `@GetMapping` – Handles HTTP GET requests.
- `@PostMapping` – Handles HTTP POST requests.
- `@PutMapping` – Handles HTTP PUT requests.
- `@DeleteMapping` – Handles HTTP DELETE requests.

Example

Java

```java
@RestController // Marks this class as a REST Controller.
@RequestMapping("/members") // Base URL for all endpoints in this
controller.
public class MemberController {

    private final MemberService memberService;

    // Spring injects the service bean through constructor injection.
    public MemberController(MemberService memberService) {
        this.memberService = memberService;
    }
```

```
    // Handles GET /members
    @GetMapping
    public List<MemberResponse> getAllMembers() {
        return memberService.getAllMembers();
    }

    // Handles GET /members/{id}
    @GetMapping("/{id}")
    public MemberResponse getMember(@PathVariable Long id) {
        return memberService.getMember(id);
    }

    // Handles POST /members
    @PostMapping
    public ResponseEntity<MemberResponse> createMember(
            @RequestBody CreateMemberRequest request) {
        MemberResponse response = memberService.create(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    // Handles PUT /members/{id}
    @PutMapping("/{id}")
    public MemberResponse updateMember(
            @PathVariable Long id,
            @RequestBody UpdateMemberRequest request) {
        return memberService.update(id, request);
    }

    // Handles DELETE /members/{id}
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteMember(@PathVariable Long id) {
        memberService.delete(id);
        return ResponseEntity.noContent().build();
    }
}
```

The `@RestController` annotation tells Spring that this class is responsible for handling REST API requests. The `@RequestMapping("/members")` annotation defines the base URL, so every endpoint in this controller starts with `/members`.

Spring provides specialized mapping annotations to handle different HTTP methods. `@GetMapping` is used to retrieve data, `@PostMapping` is used to create a new resource, `@PutMapping` is used to update

an existing resource, and `@DeleteMapping` is used to remove a resource. These annotations make the controller easier to read because the HTTP method is immediately clear from the annotation itself.

When a client sends a request, Spring automatically routes it to the appropriate controller method. Parameters in the URL, such as the member ID, are mapped using `@PathVariable`, while the request body is converted into a Java object using `@RequestBody`. After processing the request, the returned Java object is automatically serialized into JSON and sent back to the client as the HTTP response.

Notice that the controller itself contains very little business logic. Its responsibility is to receive the request, extract the required input, delegate the processing to the service layer, and return the response. Keeping controllers lightweight improves maintainability and follows the layered architecture recommended for Spring Boot applications.

Real-world usage

Almost every Spring Boot application follows this pattern. A controller exposes CRUD operations using the appropriate mapping annotations, delegates the request to the service layer, and returns the response to the client. This separation of responsibilities makes the application easier to maintain, test, and extend as new APIs are added.

Request / Response Handling

Spring Boot automatically converts incoming JSON requests into Java objects and serializes Java objects back into JSON responses using Jackson.

Common annotations used for request handling include:

- `@RequestBody` – Maps the request payload to a Java object.
- `@PathVariable` – Retrieves values from the URL path.
- `@RequestParam` – Retrieves query parameters.
- `ResponseEntity` – Provides control over HTTP status codes, headers, and response body.

Using **DTOs (Data Transfer Objects)** instead of directly exposing entity classes helps maintain a clear separation between the API contract and the persistence layer.

Example 1 - Using `@PathVariable`

```java
// Handles GET /members/101
@GetMapping("/{id}")
public MemberResponse getMember(

    // Extracts the value from the URL path.
    @PathVariable Long id) {

    return memberService.getMember(id);
```

```
}
```

The `@PathVariable` annotation is used to extract values directly from the URL path. In this example, when a client sends a request such as `GET /members/101`, Spring automatically reads the value `101`, converts it into a `Long`, and passes it as the `id` parameter to the controller method. This approach is commonly used when identifying a specific resource, such as retrieving, updating, or deleting an employee, member, or product.

Example 2 – Using `@RequestParam`

```java
// Handles GET /members/search?department=Engineering
@GetMapping("/search")
public List<MemberResponse> searchMembers(

    // Reads the query parameter from the request URL.
    @RequestParam String department) {

    return memberService.searchByDepartment(department);
}
```

The `@RequestParam` annotation is used to read query parameters from the request URL. Unlike `@PathVariable`, which is part of the URL path, query parameters are appended after a question mark (`?`). In this example, when the client sends `GET /members/search?department=Engineering`, Spring extracts the value `Engineering` and passes it to the controller method. Query parameters are typically used for filtering, searching, sorting, or pagination.

Example 3 – Using `@RequestBody`

```java
@PostMapping
public ResponseEntity<MemberResponse> createMember(

    // Converts the incoming JSON request into a Java object.
    @RequestBody CreateMemberRequest request) {

    // Delegates the request to the service layer.
    MemberResponse response = memberService.create(request);

    // Returns HTTP 201 (Created) along with the created resource.
```

```
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

The `@RequestBody` annotation is used when the client sends data in the request body, typically in JSON format. Spring Boot automatically deserializes the incoming JSON into a `CreateMemberRequest` object using the Jackson library. The controller then forwards this object to the service layer, where the business logic is executed. Finally, the controller returns a `ResponseEntity`, allowing it to specify both the response body and the appropriate HTTP status code. In this case, `201 Created` indicates that a new resource has been successfully created.

Annotation	Used for	Example
`@PathVariable`	Reading values from the URL path	`/members/101`
`@RequestParam`	Reading query parameters	`/members/search?department=Engineering`
`@RequestBody`	Reading JSON/XML request payload	`POST /members`

DTOs and Validation

DTOs are used to transfer data between the client and the application while preventing unnecessary exposure of internal domain models. They also provide a convenient place to perform input validation.

Spring Boot integrates with **Jakarta Bean Validation**, allowing developers to validate incoming requests using annotations such as:

- `@NotNull`
- `@NotBlank`
- `@Size`
- `@Email`

- `@Pattern`

The `@Valid` annotation triggers validation before the request reaches the business logic.

Example

Java

```java
// DTO used to receive data from the client while creating a new member.
public class CreateMemberRequest {

    // Name should not be null or blank.
    @NotBlank(message = "Name is required")
    private String name;

    // Email should be in a valid email format.
    @Email(message = "Invalid email address")
    private String email;

    // Age must be greater than or equal to 18.
    @Min(value = 18, message = "Age must be at least 18")
    private Integer age;
}
```

Java

```java
@PostMapping
public ResponseEntity<MemberResponse> createMember(

    // Triggers Bean Validation before the request reaches the controller
logic.
    @Valid @RequestBody CreateMemberRequest request) {

    MemberResponse response = memberService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}
```

In most REST APIs, the client sends input that must be validated before it is processed. Instead of writing manual validation logic inside the controller or service, Spring Boot integrates with **Jakarta Bean Validation** to perform these checks automatically.

In the above example, the validation rules are defined directly on the fields of the `CreateMemberRequest` DTO. The `@NotBlank` annotation ensures that the `name` field is neither null nor empty, while `@Email` verifies that the email follows a valid format. Similarly, `@Min` ensures that the age is

greater than or equal to the specified value. These annotations make the validation rules easy to understand and keep them close to the data they validate.

The `@Valid` annotation instructs Spring Boot to validate the request object before executing the controller method. When a client sends a request, Spring first converts the JSON payload into a `CreateMemberRequest` object. It then evaluates all the validation annotations present in the DTO. If every validation rule passes, the request proceeds to the business logic. However, if any validation fails, Spring immediately returns a **400 Bad Request** response containing the validation errors, and the controller method is never executed.

This approach keeps controllers clean by removing manual validation code and ensures that only valid data reaches the service layer.

Benefits

- Ensures data integrity.
- Reduces manual validation logic.
- Returns meaningful validation errors to API consumers.

Exception Handling

A robust REST API should provide consistent and meaningful error responses. Instead of handling exceptions in every controller method, Spring Boot allows centralized exception handling using @RestControllerAdvice.

Common scenarios include:

- Resource not found
- Invalid request data
- Business validation failures
- Unexpected server errors

Example

```java
// Handles exceptions thrown from controllers across the application.
@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handles MemberNotFoundException specifically.
    @ExceptionHandler(MemberNotFoundException.class)
    public ResponseEntity<String> handleMemberNotFound(
            MemberNotFoundException ex) {

        // Returns HTTP 404 along with the exception message.
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
```

```
                .body(ex.getMessage());
    }
}
```

In a REST API, exceptions can occur for various reasons, such as requesting a resource that does not exist, submitting invalid data, or encountering unexpected server errors. If every controller method handled these exceptions individually using `try-catch` blocks, the code would become repetitive and difficult to maintain.

Spring Boot provides the `@RestControllerAdvice` annotation to centralize exception handling across the entire application. Instead of writing exception handling logic in every controller, developers can define it once inside a global exception handler. Whenever a controller throws an exception, Spring automatically checks whether a matching `@ExceptionHandler` method exists and delegates the exception to that method.

In the above example, if the service layer throws a `MemberNotFoundException`, Spring invokes the `handleMemberNotFound()` method automatically. The method returns an HTTP **404 (Not Found)** response along with the exception message. This ensures that all APIs return a consistent error response without duplicating exception handling logic across multiple controllers.

Using centralized exception handling improves code readability, keeps controllers focused on request processing, and provides a consistent experience for API consumers.

Best Practices

- Keep controllers focused on request handling only.
- Use DTOs for request and response payloads.
- Validate input using Bean Validation.
- Handle exceptions centrally using @RestControllerAdvice.
- Return appropriate HTTP status codes and meaningful error messages.

Data Access

Most enterprise applications interact with a database to store and retrieve information. Spring Boot simplifies database access through **Spring Data JPA**, which provides a high-level abstraction over the Java Persistence API (JPA). Instead of writing boilerplate SQL and DAO classes, developers can focus on defining entities and repository interfaces while Spring Data handles the implementation of common database operations.

A typical data access flow in a Spring Boot application is:

Controller → Service → Repository → Database

This layered approach promotes separation of concerns, making applications easier to maintain, test, and extend.

Spring Data JPA

Spring Data JPA is a Spring module that simplifies data persistence using the Java Persistence API (JPA). It provides ready-to-use repository interfaces that eliminate the need to write repetitive CRUD operations.

By extending interfaces such as JpaRepository, developers automatically get methods for:

- Saving records
- Retrieving records
- Updating records
- Deleting records
- Pagination and sorting

Example

```java
// Repository interface for performing database operations on Member entities.
public interface MemberRepository extends JpaRepository<Member, Long> {

    // Spring automatically generates the implementation for this method
    // based on the method name.
    Optional<Member> findByEmail(String email);
}
```

`JpaRepository` is one of the most commonly used interfaces provided by Spring Data JPA. By extending this interface, developers automatically gain access to a wide range of CRUD operations without writing any implementation code. Methods such as `save()`, `findById()`, `findAll()`, `deleteById()`, pagination, and sorting are already available out of the box.

In the above example, the `MemberRepository` interface extends `JpaRepository<Member, Long>`. Here, `Member` represents the entity that will be managed, while `Long` represents the data type of its primary key. During application startup, Spring detects this repository interface and automatically creates its implementation. Developers only define the contract, while Spring generates the actual implementation at runtime.

The custom method `findByEmail(String email)` demonstrates another powerful feature of Spring Data JPA called **Query Method Derivation**. Instead of writing SQL queries manually, Spring analyzes the method name, understands that it needs to search using the `email` field, and automatically generates the required query. This significantly reduces boilerplate code and improves developer productivity.

Benefits

- Reduces boilerplate code.
- Simplifies database interactions.
- Supports pagination and sorting.

- Enables query generation using method names.

Entities and Repositories

An **Entity** represents a table in the database. Each instance of the entity corresponds to a row in that table. Spring Boot uses JPA annotations to define how Java classes are mapped to database tables.

Common annotations include:

- `@Entity` – Marks a class as a database entity.
- `@Id` – Specifies the primary key.
- `@GeneratedValue` – Defines how the primary key is generated.
- `@Column` – Maps a field to a database column.

Example

```java
// Marks this class as a JPA entity.
// Each object of this class represents a row in the database table.
@Entity
public class Member {

    // Marks this field as the primary key.
    @Id

    // Automatically generates the primary key value.
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // Maps to the 'name' column in the database table.
    private String name;

    // Maps to the 'email' column in the database table.
    private String email;
}
```

An **Entity** is a Java class that represents a table in the database. Each object of the entity corresponds to a single row, while each field represents a column in that table. Instead of writing SQL statements to map Java objects to database records, JPA uses annotations to understand how the class should be persisted.

In the above example, the `@Entity` annotation tells Spring Data JPA that the `Member` class should be managed as a database entity. During application startup, JPA scans for all classes annotated with `@Entity` and prepares the necessary metadata required to interact with the database.

The `@Id` annotation identifies the primary key of the entity, while `@GeneratedValue(strategy = GenerationType.IDENTITY)` instructs the database to generate the primary key automatically

whenever a new record is inserted. As a result, developers do not need to assign the ID manually before saving a new entity.

Once the entity is defined, the repository layer can directly perform operations such as saving, retrieving, updating, and deleting records without requiring developers to write SQL queries for common CRUD operations.

Best Practice

Keep database operations inside repository classes and avoid writing persistence logic directly in controllers.

CRUD Operations and Transactions

Most applications perform **CRUD (Create, Read, Update, Delete)** operations. Spring Data JPA provides built-in methods such as:

- save()
- findById()
- findAll()
- deleteById()

These methods cover the majority of day-to-day database interactions.

For operations involving multiple database updates, Spring provides **transaction management** through the @Transactional annotation. A transaction ensures that either all operations complete successfully or all changes are rolled back if an error occurs.

Example

```java
@Service
public class MemberService {

    private final MemberRepository repository;

    public MemberService(MemberRepository repository) {
        this.repository = repository;
    }

    // Executes all database operations within a single transaction.
    @Transactional
    public Member createMember(Member member) {

        // Saves the member to the database.
        return repository.save(member);
```

```
    }
}
```

Most business operations involve one or more database interactions. For example, creating an employee may require inserting a new employee record, updating an audit table, and recording an activity log. If one of these operations succeeds while another fails, the database can be left in an inconsistent state.

The `@Transactional` annotation ensures that all database operations within the method are executed as a single transaction. When the method starts, Spring opens a transaction. If every operation completes successfully, the transaction is committed and all changes are permanently saved to the database. However, if an exception occurs during execution, Spring automatically rolls back the transaction, restoring the database to its previous state.

In the above example, the `repository.save(member)` operation is executed within a transaction. Although only a single database operation is shown here, the same transaction can include multiple repository calls. This guarantees that either all operations succeed together or none of them are applied, maintaining data consistency.

Best Practices

- Keep transaction boundaries at the service layer.
- Use repository interfaces for database access instead of writing SQL wherever possible.
- Separate business logic from persistence logic.
- Use derived query methods before writing custom queries.

Configuration & Profiles

Application configuration plays a vital role in making Spring Boot applications flexible and environment-independent. Instead of hardcoding values such as database credentials, server ports, or API keys, Spring Boot externalizes configuration so that the same application can run across different environments with minimal changes.

Spring Boot supports multiple configuration sources, including property files, YAML files, environment variables, and command-line arguments. This approach makes applications easier to maintain, deploy, and scale.

Properties / YAML

Spring Boot allows application configuration using either **application.properties** or **application.yml** files located under the src/main/resources directory.

Both formats serve the same purpose, and the choice depends on team preference.

Example - application.properties

```
None
server.port=8080
spring.application.name=spring-boot-employee-management-demo
```

Example - application.yml

```
None
server:
  port: 8080

spring:
  application:
    name: spring-boot-employee-management-demo
```

YAML is generally preferred for hierarchical configurations, while properties files provide a simpler key-value format.

Best Practice

Use a consistent configuration format across the project to improve readability and maintainability.

Externalized Configuration

One of Spring Boot's strengths is its ability to externalize configuration, allowing applications to adapt to different environments without changing the source code.

Configuration values can be provided through:

- application.properties or application.yml
- Environment variables
- JVM system properties
- Command-line arguments

Spring Boot provides multiple ways to access configuration values within the application.

For simple values, the @Value annotation can be used.

```
Java
// Reads the value of 'server.port' from application.properties or
application.yml.
@Value("${server.port}")
private String serverPort;
```

For related configuration properties, @ConfigurationProperties offers a cleaner and type-safe approach.

```java
// Binds all properties with the prefix 'app'
// to this Java class.
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String environment;

    // Getters and Setters
}
```

Spring Boot allows application configuration to be stored outside the source code so that the same application can run in different environments without requiring code changes. Instead of hardcoding values such as server ports, database credentials, API keys, or feature flags, these values are defined in configuration files like `application.properties` or `application.yml`.

The `@Value` annotation is the simplest way to read an individual configuration property. In the above example, Spring reads the value of `server.port` from the configuration file and injects it into the `serverPort` variable during application startup. This approach is suitable when only one or two configuration values need to be accessed.

When multiple related configuration properties need to be managed, `@ConfigurationProperties` provides a cleaner and more maintainable approach. Instead of reading each property individually, Spring automatically maps all properties with the specified prefix into a Java object. This keeps related configuration organized, improves readability, and provides type safety.

By externalizing configuration, developers can use the same application build across development, testing, and production environments while changing only the configuration values, without modifying or recompiling the application.

Spring Profiles

Applications often require different configurations for development, testing, and production environments. **Spring Profiles** allow developers to maintain separate configurations while using the same application code.

Common profiles include:

- **dev** – Development environment
- **test** – Testing environment
- **prod** – Production environment

Each profile can have its own configuration file, such as:

- `application-dev.yml`
- `application-test.yml`

- `application-prod.yml`

The active profile can be configured using:

```
None
spring.profiles.active=dev
```

Beans can also be loaded conditionally using the `@Profile` annotation.

```java
Java
// This configuration class is loaded only when
// the 'dev' profile is active.
@Configuration
@Profile("dev")
public class DevelopmentConfig {

    // Development-specific bean definitions.
}
```

Spring Profiles allow an application to use different configurations for different environments without changing the source code. In real-world projects, the configuration required during development is often different from the one used in testing or production. For example, a development environment might use a local database and detailed logging, while the production environment connects to a production database with minimal logging.

Spring Boot manages this by allowing multiple profile-specific configuration files, such as `application-dev.yml`, `application-test.yml`, and `application-prod.yml`. When the application starts, Spring first loads the common configuration from `application.yml` and then loads the configuration file that matches the active profile. Properties defined in the active profile override the common configuration wherever required.

The `@Profile` annotation can also be applied to beans and configuration classes. In the above example, the `DevelopmentConfig` class is created only when the `dev` profile is active. If the application is started with the `prod` profile, this configuration class is ignored, allowing different beans or settings to be used for different environments without changing the application code.

Best Practices

- Keep common configuration in the base configuration file.
- Store sensitive information such as passwords and API keys in environment variables or secure secret management solutions.
- Use profiles to separate development, testing, and production configurations.
- Prefer `@ConfigurationProperties` for managing related configuration values.

Testing

Testing is an essential part of software development that helps ensure applications behave as expected and continue to work correctly as new features are introduced. Spring Boot provides excellent support for testing through the **Spring Boot Test** framework, along with popular testing libraries such as **JUnit** and **Mockito**.

A well-tested application improves code quality, reduces production issues, and gives developers confidence when making changes or refactoring existing code.

Unit Testing and Integration Testing

Spring Boot primarily supports two types of testing:

Unit Testing

Unit testing focuses on testing individual classes or methods in isolation. Dependencies such as repositories or external services are typically mocked so that only the business logic is tested.

Unit tests are:

- Fast to execute
- Independent of external systems
- Easy to maintain
- Useful for validating business logic

Integration Testing

Integration testing verifies that multiple components work together correctly. Unlike unit tests, integration tests load the Spring Application Context and validate interactions between controllers, services, repositories, and the database.

Spring Boot provides the @SpringBootTest annotation to bootstrap the complete application context during testing.

Example

```java
// Loads the complete Spring Boot application context
// for integration testing.
@SpringBootTest
class EmployeeManagementApplicationTests {

    @Test
    void contextLoads() {

        // Verifies that the Spring Application Context
```

```
        // starts successfully without any configuration errors.
    }
}
```

Spring Boot supports both **Unit Testing** and **Integration Testing**, each serving a different purpose. Unit tests focus on testing a single class or method in isolation, while integration tests verify that multiple components of the application work together correctly.

The `@SpringBootTest` annotation is used to load the complete Spring Boot application context. When this annotation is present, Spring starts the application in a test environment by creating all required beans, applying configuration, and resolving dependencies, just as it would during normal application startup.

In the above example, the `contextLoads()` test does not contain any assertions. Its purpose is simply to verify that the application starts successfully without any configuration, bean creation, or dependency injection issues. If Spring fails to create the application context due to missing beans, incorrect configuration, or startup errors, this test fails immediately, helping developers identify configuration problems early in the development cycle.

Since loading the entire application context requires more resources, integration tests are generally slower than unit tests. Therefore, they should be used to validate interactions between different layers of the application, while unit tests should continue to verify individual classes and business logic in isolation.

REST API Testing

REST APIs should be tested to ensure they return the expected responses, status codes, and error messages.

Spring Boot provides **MockMvc**, which allows developers to test controller endpoints without deploying the application to an external server.

REST API testing typically verifies:

- HTTP status codes
- Request validation
- Response payload
- Exception handling
- API behavior for different scenarios

Example

Java

```
@WebMvcTest(MemberController.class)
```

```
class MemberControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private MemberService memberService;

    @Test
    void shouldReturnMemberWhenIdExists() throws Exception {

        // Mock the service response.
        when(memberService.getMember(1L))
                .thenReturn(new MemberResponse(1L, "Sachin",
"sachin@example.com"));

        // Perform a GET request and verify the response.
        mockMvc.perform(get("/members/1"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.name").value("Sachin"));
    }
}
```

Spring Boot provides **MockMvc** to test REST APIs without starting an actual web server. Instead of deploying the application and sending HTTP requests over the network, MockMvc simulates HTTP requests directly against the Spring MVC layer. This makes controller tests fast, reliable, and easy to execute as part of the build process.

In the above example, the `@WebMvcTest` annotation loads only the web layer required for testing the `MemberController`. The service layer is mocked using `@MockBean`, allowing the controller to be tested independently without accessing the database or executing business logic.

The `mockMvc.perform()` method simulates an HTTP GET request to `/members/1`. The `andExpect()` methods are then used to verify different aspects of the response, such as the HTTP status code and the response body. In this example, the test verifies that the API returns **HTTP 200 (OK)** and that the returned JSON contains the expected member name.

Using MockMvc allows developers to validate request mappings, request validation, response payloads, exception handling, and HTTP status codes, ensuring that REST APIs behave as expected before they are deployed.

Best Practice

Write tests for both successful and failure scenarios, including invalid input and resource-not-found cases.

Mocking Dependencies

Many components depend on external systems such as databases, messaging services, or third-party APIs. During unit testing, these dependencies should be replaced with **mock objects** to isolate the component being tested.

Spring Boot commonly uses **Mockito** for mocking dependencies.

Example

```java
@ExtendWith(MockitoExtension.class)
class MemberServiceTest {

    // Creates a mock implementation of MemberRepository.
    @Mock
    private MemberRepository repository;

    // Creates MemberService and injects the mocked repository.
    @InjectMocks
    private MemberService service;

    @Test
    void shouldReturnMemberWhenIdExists() {

        // Configure the mock to return predefined data.
        when(repository.findById(1L))
                .thenReturn(Optional.of(new Member(1L, "Sachin",
"sachin@example.com")));

        // Invoke the business method.
        MemberResponse response = service.getMember(1L);

        // Verify the expected result.
        assertEquals("Sachin", response.getName());
    }
}
```

Unit tests should focus on testing the business logic of a class without depending on external systems such as databases, messaging services, or third-party APIs. Instead of interacting with these real dependencies, Mockito allows developers to replace them with mock objects that simulate their behavior.

In the above example, the `@Mock` annotation creates a mock implementation of `MemberRepository`. This mock does not communicate with an actual database. Instead, its behavior is controlled by the test using the `when(...).thenReturn(...)` statement. Whenever the service calls

`repository.findById(1L)`, Mockito returns the predefined `Member` object instead of executing a real database query.

The `@InjectMocks` annotation creates an instance of `MemberService` and automatically injects the mocked repository into it. As a result, the test executes only the business logic inside the service while keeping external dependencies isolated. This makes unit tests faster, more reliable, and easier to maintain because their outcome does not depend on the availability of databases or other external resources.

Best Practices

- Write unit tests for business logic and integration tests for application workflows.
- Keep tests independent and repeatable.
- Mock external dependencies instead of relying on actual services or databases.
- Cover both positive and negative test scenarios.
- Execute tests regularly as part of the build or CI/CD pipeline.

Gem

- Spring Boot

Demo Code Links

- **Employee Management API** – A sample Spring Boot application demonstrating layered architecture, REST APIs, DTOs, validation, exception handling, configuration, profiles, testing, and an in-memory repository implementation. (Zip file)

Reference material

- Baeldung – Learn Spring Boot Series
- Spring Boot Official Documentation
- Spring Initializr

Advanced reading material (good to have)

- Spring Security Documentation
- Spring Boot Actuator Documentation
- Spring Cloud Documentation
- Baeldung – Spring Security Tutorials